KUBERNETES CKAD

1. Core Concepts

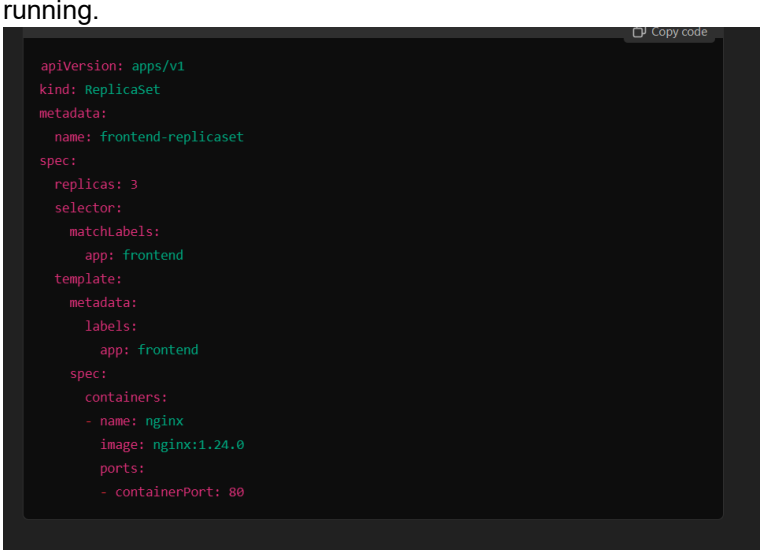
```yaml
apiVersion: v1          # String values
kind: Pod
metadata:               # dictionary (2 spaces for tab-ish)
  name: myapp-pod
  labels:
    app: mypod
    type: front-end
spec:
  containers:                 # List/Array (of containers)
    - name: nginx-container   # dash indicates it is item of list
      image: nginx
```

 kubectl run busybox --image=busybox --command --restart=Never -it --rm -- env - Run a busybox Pod to execute the env command and immediately delete it after completion.
 kubectl run busybox --image=busybox --command --restart=Never -- env - Run a busybox Pod to execute the env command.
 kubectl logs busybox - Get logs of the busybox Pod.
 kubectl run busybox --image=busybox --restart=Never --dry-run=client -o yaml --command -- env > envpod.yaml - Generate a YAML manifest for a busybox Pod that runs the env command.
 kubectl apply -f envpod.yaml - Apply the YAML manifest to create the Pod.
 kubectl logs busybox - Get the logs of the busybox Pod created with YAML.
 kubectl create namespace myns -o yaml --dry-run=client - Generate YAML for a new namespace named myns without creating it.
 kubectl create quota myrq --hard=cpu=1,memory=1G,pods=2 --dry-run=client -o yaml - Generate YAML for a ResourceQuota with specific limits without creating it.
 kubectl get po --all-namespaces - List Pods across all namespaces.
 kubectl run nginx --image=nginx --restart=Never --port=80 - Create an nginx Pod and expose traffic on port 80.
 kubectl set image pod/nginx nginx=nginx:1.24.0 - Update the nginx Pod's image to version 1.24.0.
 kubectl get po nginx -o jsonpath='{.spec.containers[].image}{"\n"}' - Check the image of the nginx Pod.
 kubectl get po -o wide - Get detailed information, including IPs, of all Pods.
 kubectl run busybox --image=busybox --rm -it --restart=Never -- wget -O- 10.1.1.131:80 - Use a busybox Pod to wget the / endpoint of an nginx Pod by its IP.
 kubectl get po nginx -o yaml - Get the YAML definition of the nginx Pod.
 kubectl describe po nginx - Get detailed information and potential issues with the nginx Pod.
 kubectl logs nginx - Get logs of the nginx Pod.
 kubectl logs nginx -p - Get logs of the nginx Pod's previous instance if it restarted.
 kubectl exec -it nginx -- /bin/sh - Execute a shell in the nginx Pod.
 kubectl run busybox --image=busybox -it --restart=Never -- echo 'hello world' - Create a busybox Pod that echoes "hello world" and exits.
 kubectl run busybox --image=busybox -it --rm --restart=Never -- echo 'hello world' - Create a busybox Pod, echo "hello world", and delete it automatically after completion.

 kubectl run nginx --image=nginx --restart=Never --env=var1=val1 - Create an nginx Pod with an environment variable var1=val1.
 kubectl exec -it nginx -- env - Check the environment variables inside the nginx Pod.
 kubectl describe po nginx | grep val1 - Verify the environment variable in the Pod description.
 kubectl run nginx --image=nginx --restart=Never --env=var1=val1 -it --rm -- env - Create a temporary nginx Pod with an environment variable and check its value.

ReplicaSet

 kubectl apply -f replicaset.yaml - Create or update a ReplicaSet from a YAML file.
 kubectl get rs - List all ReplicaSets in the current namespace.
 kubectl describe rs frontend-replicaset - Get detailed information about a specific ReplicaSet.
 kubectl get pods -l app=frontend - List Pods managed by a ReplicaSet using a label selector.
 kubectl scale --replicas=5 rs/frontend-replicaset - Scale a ReplicaSet to 5 replicas imperatively.
 kubectl scale --replicas=5 -f replicaset.yaml - Scale a ReplicaSet defined in a YAML file.
 kubectl delete rs frontend-replicaset - Delete a ReplicaSet and its Pods.
 kubectl delete rs frontend-replicaset --cascade=orphan - Delete a ReplicaSet but keep its Pods running.



Deployments

 kubectl apply -f deployment.yaml - Create or update a Deployment.
 kubectl get deployments - List all Deployments in the current namespace.
 kubectl describe deployment myapp-deployment - Get detailed information about a specific Deployment.
 kubectl rollout status deployment myapp-deployment - Check the status of a Deployment rollout.
 kubectl set image deployment myapp-deployment nginx-container=nginx:1.24 --record - Perform a rolling update by changing the image of the Deployment.
 kubectl rollout undo deployment myapp-deployment - Roll back the Deployment to the previous revision.
 kubectl rollout pause deployment myapp-deployment - Pause the Deployment rollout.
 kubectl rollout resume deployment myapp-deployment - Resume a paused Deployment rollout.

 kubectl scale deployment myapp-deployment --replicas=5 - Scale the Deployment to 5 replicas.
 kubectl rollout history deployment myapp-deployment - View the revision history of a Deployment.

Resource Quota sample

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-quota
  namespace: dev
spec:
  hard:
    pods: "10"              # Maximum number of Pods allowed in the namespace
    requests.cpu: "4"       # Total CPU requests for all Pods in the namespace
    requests.memory: 5Gi    # Total memory requests for all Pods in the namespace
    limits.cpu: "10"        # Maximum CPU limits for all Pods in the namespace
    limits.memory: 10Gi     # Maximum memory limits for all Pods in the namespace
```

Practice on imperative commands

Pod Commands

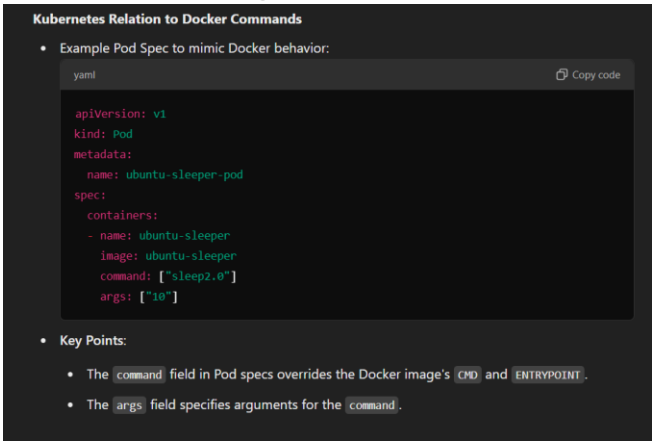
- kubectl expose pod redis --port=6379 --name redis-service --dry-run=client -o yaml - Create a ClusterIP Service for a Pod exposing port 6379 and generate its YAML.
- kubectl create service clusterip redis --tcp=6379:6379 --dry-run=client -o yaml - Generate a ClusterIP Service YAML for port 6379 with default selectors.
- kubectl expose pod nginx --port=80 --name nginx-service --type=NodePort --dry-run=client -o yaml - Create a NodePort Service YAML for a Pod exposing port 80.
- kubectl create service nodeport nginx --tcp=80:80 --node-port=30080 --dry-run=client -o yaml - Generate a NodePort Service YAML exposing port 80 with a specific node port.

General Commands

- kubectl get all - List all resources (Pods, Deployments, Services, etc.) in the namespace.

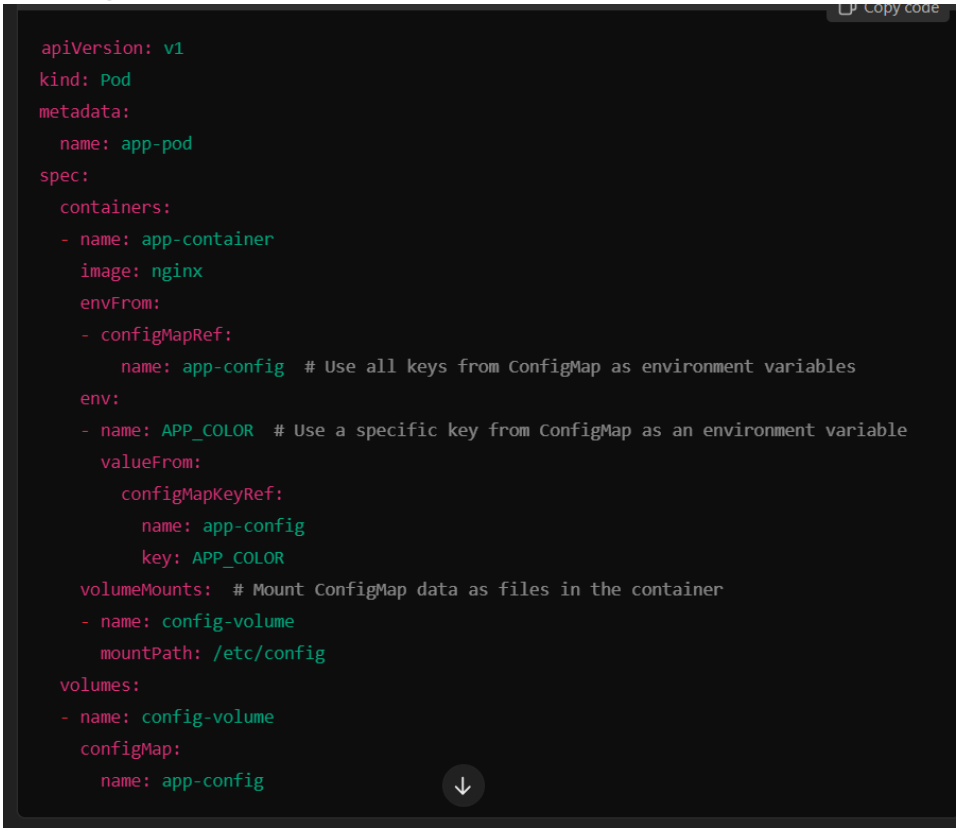
2. Configuration

Container images



- docker pull <image_name> - Pull a Docker image from a registry.
- docker images - List all downloaded images on the system.
- docker run <image_name> - Run a container from an image.
- docker run -it <image_name> - Start a container in interactive mode.
- docker ps - List running containers.
- docker ps -a - List all containers (including stopped ones).
- docker stop <container_id> - Stop a running container.
- docker rm <container_id> - Remove a container.
- docker rmi <image_name> - Remove an image.
- docker exec -it <container_id> /bin/sh - Access a running container with a shell.
- docker logs <container_id> - View logs of a running or stopped container.
- docker inspect <container_id> - Get detailed information about a container.
- docker build -t <image_name> . - Build an image from a Dockerfile in the current directory.
- docker tag <image_name> <repository_name>:<tag> - Tag an image for pushing to a registry.
- docker push <repository_name>:<tag> - Push an image to a Docker registry.
- docker run -p HOST_PORT:CONTAINER_PORT <image_name> - Map container ports to host ports.

ConfigMaps



kubectl create configmap app-config --from-literal=APP_COLOR=blue - Create a ConfigMap imperatively.

Secrets

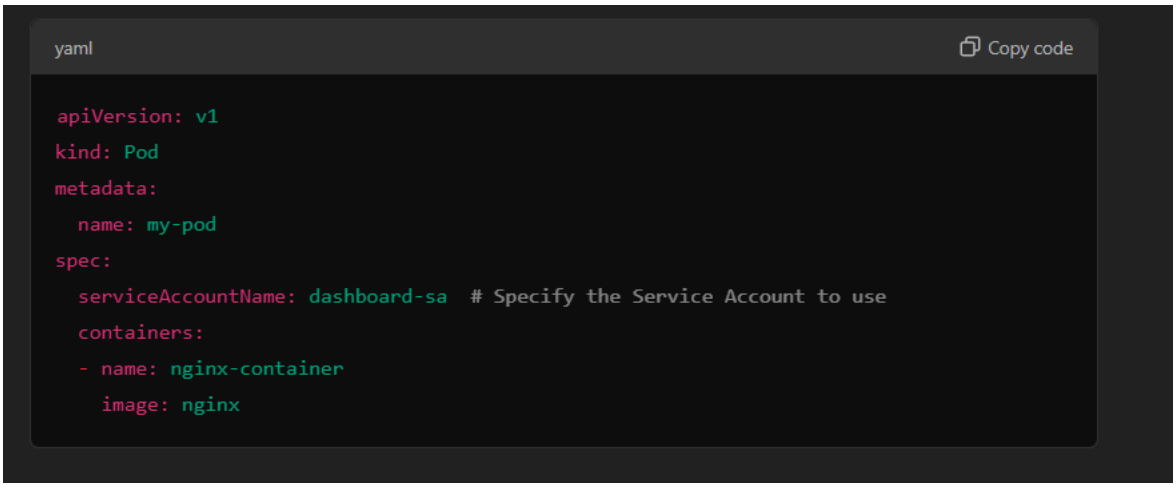
 kubectl create secret generic <secret-name> --from-literal=<key>=<value> - Create a Secret with a key-value pair.
 kubectl create secret generic <secret-name> --from-file=<file-path> - Create a Secret from a file.
 kubectl create secret generic <secret-name> --dry-run=client -o yaml - Generate a Secret YAML without creating it.
 kubectl get secrets - List all Secrets in the current namespace.
 kubectl describe secret <secret-name> - View detailed information about a Secret.
 kubectl get secret <secret-name> -o yaml - Get a Secret manifest in YAML format.
 kubectl delete secret <secret-name> - Delete a specific Secret.
 echo -n "my-secret-value" | base64 - Encode a value to Base64 for Secret YAML.
 echo -n "c2VjcmV0dmFsdWU=" | base64 --decode - Decode a Base64-encoded Secret value.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
  - name: app-container
    image: nginx
    envFrom:
    - secretRef:
        name: app-secret        # Use all keys from Secret as environment variables
    env:
    - name: APP_PASSWORD        # Use a specific key from Secret
      valueFrom:
        secretKeyRef:
          name: app-secret
          key: APP_PASSWORD
    volumeMounts:
    - name: secret-volume       # Mount Secret as files
      mountPath: /etc/secret
  volumes:
  - name: secret-volume
    secret:
      secretName: app-secret
```

Security Context
Container level overrides pod level

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: combined-security
  labels:
    app: secure-app
spec:
  securityContext:         # Pod-level security context
    runAsUser: 1000        # Default user ID for all containers in the Pod
    runAsGroup: 3000       # Default group ID for all containers
    fsGroup: 2000          # Group ID for file system ownership
  containers:
  - name: nginx-container
    image: nginx
    securityContext:       # Container-level security context (overrides Pod-level context)
      runAsUser: 2000      # Specific user ID for this container
      runAsGroup: 4000     # Specific group ID for this container
      capabilities:        # Capabilities specific to this container
        add: ["NET_ADMIN"] # Add NET_ADMIN capability
        drop: ["KILL"]     # Drop KILL capability
  - name: busybox-container
    image: busybox
    securityContext:       # Another container-level security context
      runAsUser: 3000      # Specific user ID for this container
      capabilities:
        add: ["SYS_TIME"]  # Add SYS_TIME capability
```

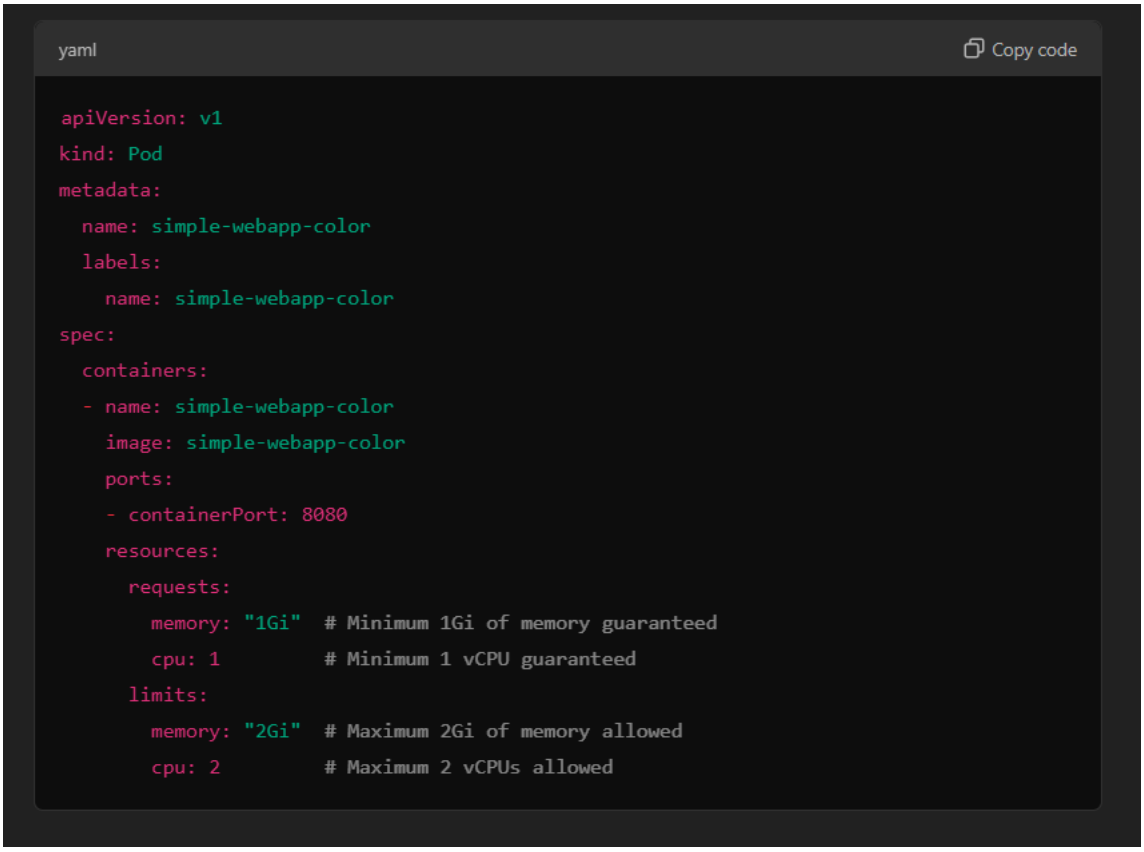
Service Accounts



```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  serviceAccountName: dashboard-sa  # Specify the Service Account to use
  containers:
  - name: nginx-container
    image: nginx
```

it creates a token and stores in a secret, to create a new token on demand use, k create token <>

Resource Requirements



```yaml
apiVersion: v1
kind: Pod
metadata:
  name: simple-webapp-color
  labels:
    name: simple-webapp-color
spec:
  containers:
  - name: simple-webapp-color
    image: simple-webapp-color
    ports:
    - containerPort: 8080
    resources:
      requests:
        memory: "1Gi"  # Minimum 1Gi of memory guaranteed
        cpu: 1         # Minimum 1 vCPU guaranteed
      limits:
        memory: "2Gi"  # Maximum 2Gi of memory allowed
        cpu: 2         # Maximum 2 vCPUs allowed
```

yaml Copy code

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: memory-resource-constraint
  namespace: default
spec:
  limits:
  - default:        # Default memory limit if not specified in Pods
      memory: 1Gi
    defaultRequest: # Default memory request if not specified in Pods
      memory: 1Gi
    max:            # Maximum memory limit for Pods
      memory: 2Gi
    min:            # Minimum memory request for Pods
      memory: 500Mi
    type: Container
```

Taints and Tolerations

Taints are set on Nodes, Tolerations are set on Pods

Taints

Taints are applied to nodes to repel Pods that do not tolerate them. They define restrictions on which Pods can run on a node.

- **Command to Add Taint:**

bash Copy code

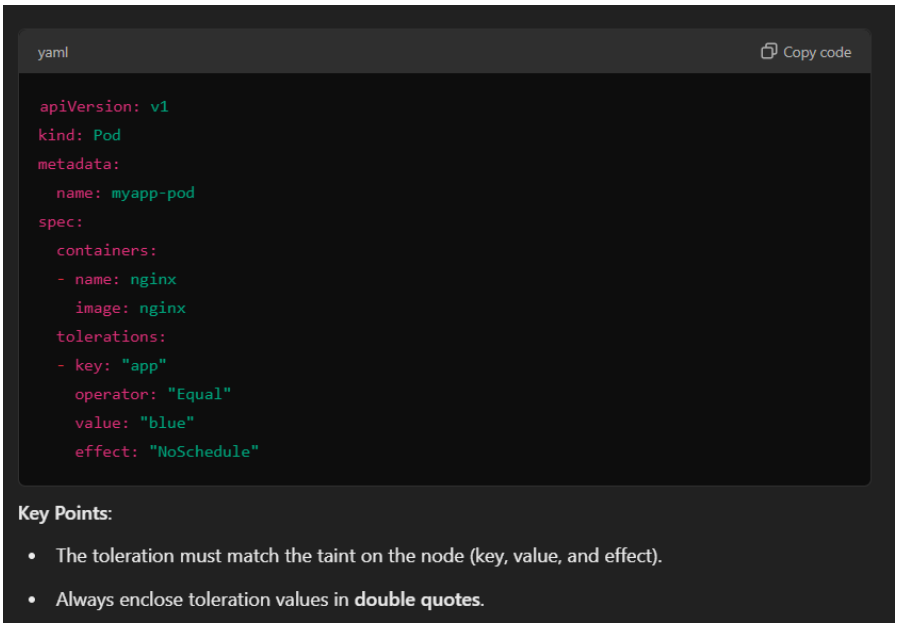
```bash
kubectl taint nodes <node-name> <key>=<value>:<effect>
```

Examples:

- `kubectl taint nodes node1 app=blue:NoSchedule` - Prevent Pods without the appropriate toleration from being scheduled on `node1`.
- **Taint Effects:**
  - `NoSchedule`: Pods without tolerations won't be scheduled.
  - `PreferNoSchedule`: Scheduler avoids the node but doesn't guarantee exclusion.
  - `NoExecute`: Evicts existing Pods without tolerations and prevents new ones from being scheduled.

 kubectl taint nodes <node-name> <key>=<value>:<effect> - Add a taint to a node.
Example: kubectl taint nodes node1 app=blue:NoSchedule
 kubectl taint nodes <node-name> <key>:<effect>- - Remove a taint from a node.
Example: kubectl taint nodes node1 app=blue:NoSchedule-
 kubectl describe node <node-name> | grep Taint - View taints on a specific node.

 kubectl taint nodes kubemaster node-role.kubernetes.io/master:NoSchedule- - Untaint the master node to allow Pods to be scheduled.



Node Selectors

- For assigning specifically which node a pod should be scheduled. Likewise for Node Affinity
- kubectl label nodes <node-name> <label-key>=<label-value> - Add a label to a node. Example: kubectl label nodes node01 size=Large
- kubectl label nodes <node-name> <label-key>- - Remove a label from a node. Example: kubectl label nodes node01 size-

Node Affinity

- Finer control than Node Selectors

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
  - name: nginx
    image: nginx
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: size
            operator: In
            values:
            - Large
            - Medium
```

 requiredDuringSchedulingIgnoredDuringExecution: Pods are only scheduled on matching nodes; changes after scheduling do not evict Pods.

 preferredDuringSchedulingIgnoredDuringExecution: Scheduler prefers matching nodes but can schedule elsewhere; changes after scheduling do not evict Pods.

 requiredDuringSchedulingRequiredDuringExecution: Pods are only scheduled on matching nodes and are evicted if rules change (not implemented yet).

Multi-container Pods
Common Patterns:

1. Sidecar
    - A logging service for your application service
2. Adapter
    - Before sending the logs, conversion service act as adapter to harmonize log format
3. Ambassador
    - Will proxy the traffic to persist to proper environment DBs

Init Containers

Example Pod YAML with Init Containers

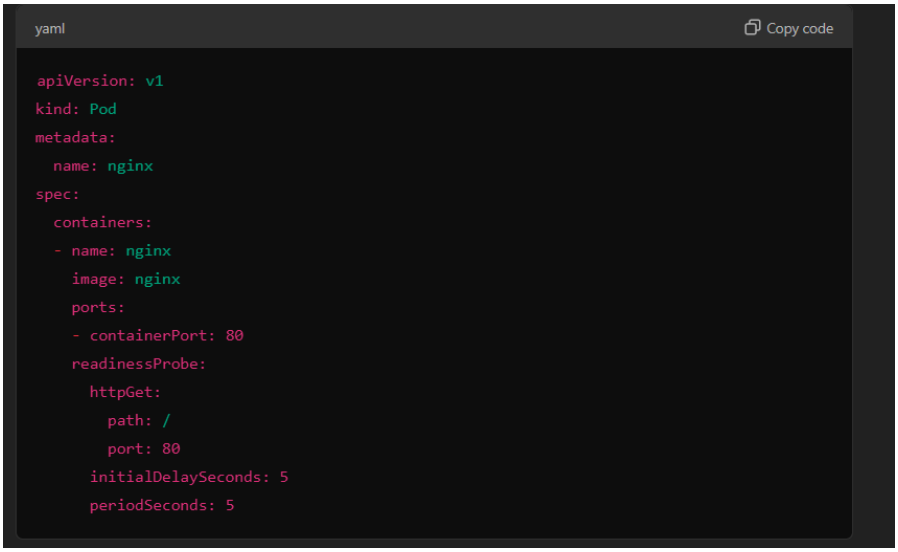
yaml Copy code

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: init-container-example
spec:
  containers:
  - name: main-app
    image: nginx
  initContainers:
  - name: init-task
    image: busybox
    command:
    - sh
    - -c
    - "echo Initializing... && sleep 5"
  - name: setup-db
    image: busybox
    command:
    - sh
    - -c
    - "echo Setting up DB && sleep 10"
```

Observability
Readiness and Liveness Probes

yaml Copy code

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx
    livenessProbe:
      exec:
        command:
        - ls
      initialDelaySeconds: 5
      periodSeconds: 5
```



Logs:

kubectl run busybox --restart=Never --image=busybox -- notexist

kubectl logs busybox # Container never started, no logs

kubectl describe pod busybox # Check the events section for errors

kubectl get events | grep -i error

kubectl delete pod busybox --force --grace-period=0

kubectl top nodes